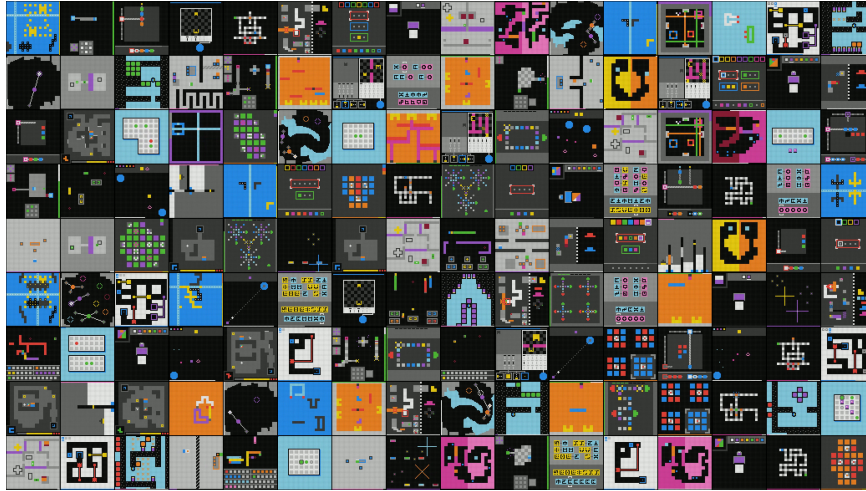


Reinforcement Learning Assignment

COMS4061A/COMS7071A

ARC-AGI-3: Interactive Reasoning Agents

Moodle due date: 27 October 2026, 23:59 SAST



1 Introduction

Intelligent agents must often act before they know how an environment works or what constitutes success. They must explore, infer useful structure, retain information, and adapt their behaviour using limited interaction. These challenges connect reinforcement learning to reasoning, planning, and generalisation.

Your task is to implement and investigate learning agents for **ARC-AGI-3**, the interactive reasoning benchmark used in [ARC Prize 2026](#). You will compare two learning algorithms, improve them through evidence-based experimentation, and communicate your findings in a concise research report.

Each group will participate as a team in both the [ARC Prize 2026 – ARC-AGI-3](#) and [ARC Prize 2026 – Paper Track](#) on Kaggle. You will also submit your report and code on Moodle. This provides an opportunity to compare your work with international submissions while keeping course assessment focused primarily on the quality of your investigation.

2 Environment

ARC-AGI-3 consists of unfamiliar interactive environments in which agents must discover rules and goals through interaction, without explicit instructions. Unlike the static input–output puzzles of earlier ARC benchmarks, this project concerns sequential decision-making in interactive games.

Start with the official [ARC-AGI-3 documentation](#) and the competition’s own starter materials. Inspect the observation format, available actions, reset behaviour, level progression, and scoring interface before choosing your agent design. Explain how your implementation represents observations or histories, actions, feedback, and termination.

Use the official competition interface for submissions. If your RL library requires Gymnasium, implement an adapter and document its semantics (the default ARC interface is not a Gymnasium environment). Record package and environment versions, and run a minimal end-to-end agent before beginning large experiments.

3 Requirements

You are allowed to work in groups of **at most 4 students**. Each group will be required to do the following for this assignment:

1. **Implement two baseline learning agents.** Use one learning algorithm covered in the course (e.g. DQN) and one learning algorithm not covered in the course (e.g. PPO). Ask a tutor if you are unsure whether your choices qualify. Explain both methods and give enough detail about the unfamiliar algorithm to demonstrate that you understand its learning objective and updates. Identify what is learned, when learning occurs, and how it affects decisions.
2. **Evaluate both baselines.** Establish a reproducible evaluation protocol before making improvements. Include a simple random-action or other inexpensive reference policy as a sanity check; it does not replace either learning algorithm.
3. **Iteratively improve each agent.** For *each* of your two algorithms, implement and evaluate at least **two substantive improvements**. Use the results of each stage to motivate the next design decision:

Baseline and evaluation $\longrightarrow$ Improvement 1 and evaluation
 $\longrightarrow$ Improvement 2 and evaluation.

This gives at least three evaluated versions per algorithm. For each change, state the observed limitation, your hypothesis, the intervention, and what the results show. A change need not increase the score to earn credit if it is well motivated, tested, and analysed.

4. **Make meaningful changes.** Suitable directions include exploration strategies, memory, learned representations, model-based planning, hierarchical skills, curriculum learning, or reward shaping. Merely changing hyperparameters is insufficient. If an idea is used for both algorithms, evaluate its effect separately for each.
5. **Compare the final agents.** Discuss their strengths, weaknesses, interaction efficiency, computational requirements, and ability to adapt to held-out environments. Include ablations that help isolate the contribution of your main improvements.
6. **Distinguish your contribution from reused work.** Cite all research, code, pretrained models, and other resources used. Explain the modifications you made and what you implemented yourselves. An unchanged existing agent or a prompted model alone does not satisfy the two-learning-algorithm requirement.

The emphasis is on the investigation. Strong marks do not require a world-leading score. Clear reasoning, sound experiments, and an honest analysis of unsuccessful ideas are valuable. Additional improvements may strengthen the relevant rubric categories, but do not automatically earn marks beyond the 100-point total.

4 Particulars and Restrictions

1. **Libraries and resources.** You are allowed to use libraries to implement and train your agents (e.g. Stable Baselines3, Hugging Face, etc). Follow the competition rules governing external data, pretrained models, internet access, compute, etc. The competition provides compute via Kaggle notebooks.
2. **Preserve the benchmark.** Training-time preprocessing, action abstractions, and reward shaping are allowed when compatible with the competition rules. Report evaluation using the unmodified official scoring procedure. Do not present shaped training return as the ARC-AGI-3 score.
3. **Prevent information leakage.** Agents must act using information permitted by the evaluation interface. Do not use hidden game state, held-out solutions, or game-specific action scripts. Separate training/development environments from held-out evaluation environments wherever the available public set permits. Disclose every environment used during development and avoid claiming generalisation from repeated testing on the same games.
4. **Specify adaptation.** State which parameters or memories persist between actions, levels, games, and evaluation runs. Distinguish offline training from online learning or adaptation during evaluation, and respect the competition's reset and information-sharing rules.
5. **Fair comparisons.** Compare agents on the same games and comparable interaction and compute budgets. Document action limits, resets, stopping criteria, checkpoint selection, and random seeds. Report training cost separately from evaluation cost. Disclose and justify any unequal budgets.
6. **Team participation.** Join both Kaggle tracks early with the same group membership and a consistent team name. Check entry and team-merger deadlines directly on Kaggle.
7. **Generative-AI tools.** These may assist your work subject to the wits academic-integrity policy. You remain responsible for understanding, verifying, citing or declaring their use as required, and for every claim and line of submitted code.

5 Evaluation

For the baseline and improved versions of both algorithms, report:

- The official ARC-AGI-3 score, with the evaluation set and scoring version identified. Distinguish local scores from Kaggle scores.
- Level/game completion and actions used, including unsuccessful runs; do not report only successful trajectories.
- Learning or adaptation curves against interaction budget, and relevant runtime, hardware, and resource use.
- Variability across at least three independent seeds for stochastic experiments, or a justified limitation where resources prevent this. Use matched evaluation conditions and show individual runs or uncertainty where feasible.
- Representative success and failure cases, supported by trajectories or replays, and evidence for the effect of the proposed improvements.

Use plots for trends and tables for compact comparisons. Explain what each result establishes and what remains uncertain. Essential evidence must appear within the report's main page limit.

The report is the primary assessment artifact. Code and supporting files are used to assess reproducibility and verify its claims. The following breakdown applies to both course codes.

Criterion	Marks	Evidence expected for full credit
Problem formulation and understanding	10	Clear problem, MDP definition (justified state/history and feedback choices), and relevant prior work, and accurate explanation of both baseline learning algorithms.
Baseline implementation and evaluation	20	Two functioning learning baselines; a clear protocol, credible initial results, and an appropriate sanity-check policy.
Iterative improvements	25	At least two substantive changes per algorithm, motivated by prior evidence, correctly implemented and evaluated, with clear attribution of effects.
Experimental rigour and analysis	25	Fair comparisons, ablations, variability, efficiency and generalisation analysis, informative failure cases, and conclusions supported by results.
Report quality and reproducibility	10	Concise, coherent writing; legible figures; correct citations; complete methods and configurations; and submission evidence.
Class leaderboard	10	Relative class performance of the eligible code submission, calculated below.
Total	100	

For each of the first five criteria, marks reflect the degree to which the evidence above is met: **80–100%** of the available marks for thorough, correct, well-supported work; **60–79%** for sound work with minor gaps; **40–59%** for partial evidence or substantial weaknesses; and **0–39%** for missing, incorrect, or largely unsupported work. These descriptors apply within each criterion.

Class leaderboard: 10 marks

The class leaderboard uses the **official Kaggle public score recorded by the Moodle deadline**, restricted to class teams. Each group must nominate one successfully evaluated submission made by the deadline, include its identifier and score evidence, and submit the matching code on Moodle (or include a public code link at the end of the abstract). Later submissions and the final worldwide ranking do not change course marks.

Let N be the number of eligible class teams and r the group's rank, with rank 1 best. Award

$$L = 10 \frac{N - r}{N - 1} \quad (N > 1).$$

If all teams tie, each receives 10 marks. Missing or invalid submissions receive 0 leaderboard marks. Valid zero-score submissions remain eligible.

6 Submission

One member must submit on behalf of the group. Include all members' names and student emails (using student numbers) in the report's author list. Submit a ZIP file named `Kaggle-team-name.zip` containing:

1. **Project report (PDF).** Use the [official RLC 2026 LaTeX template](#), linked from the [RLC submission instructions](#), and **disable the cover page**. The page limit is **eight pages of main content excluding references**, including figures, tables, and any appendices placed before the references. References and optional supplementary material after the references are excluded. The report must be self-contained within eight pages. Supplementary material cannot replace required analysis. Keep the template's fonts, margins, and spacing.

Organise the report around the problem and research question, relevant background, methods, experimental setup, iterative results and ablations, discussion of limitations, and conclusions. Include the hyperparameters and resource budgets needed to interpret the results. Write concisely: avoid generic textbook exposition, repetition, unsupported claims, and filler. Eight pages is a maximum, not a target. Use [Less is More: Recursive Reasoning with Tiny Networks](#) as a quality reference for focused scientific communication, not as an exemption from submission limits.

2. **Source code (Notebook)** You can download the Kaggle notebook for your Kaggle submission. Marks will be awarded for the quality of your code. Remove API keys and other credentials.
3. **Competition evidence.** Include the code submission identifier, score and timestamp evidence, the nominated class-leaderboard entry, and the Paper Track write-up link (plus a saved copy of the submitted text). The paper must describe the approach demonstrated by the linked code submission. Include a brief statement of each member's contribution.

Kaggle submissions and later participation

Submit working code to [ARC Prize 2026 – ARC-AGI-3](#) and your report/write-up to [ARC Prize 2026 – Paper Track](#) by the Moodle deadline. Follow each track's current submission mechanism. Since Kaggle requires a shorter write-up (<1500 words), you may prepare a compliant condensed version of the same investigation and link or attach the full report.

The [official schedule](#), checked on 15 September 2026, lists **2 November 2026** for competition submissions, **8 November 2026** for papers, and **4 December 2026** for results. Verify exact cut-off times and any changes on Kaggle. You may refine your external entries after the Moodle deadline to see how your team performs worldwide; the Moodle submission remains the version assessed for the course.

Further research opportunity. The most promising submission, if any, may be invited to work with the lecturer to refine and extend the project into a full paper for submission to the **Reinforcement Learning Journal (RLJ)**. This is optional and does not guarantee acceptance. A later RLJ submission must follow the journal's then-current requirements, including any cover-page requirement.